ZentroMart VoiceOps 2026: Technical Report

Project: AI Customer Support Agent with Voice (LLM + RAG + Gnani STT/TTS)

Name: Pritesh Pravin Purkar

GitHub Link: https://github.com/PriteshPurkar/zentromart-voiceops

1. Introduction & Architecture Overview

The objective of this capstone was to build a customer support agent for the fictional Indian e-commerce store, ZentroMart. The agent needed to answer questions based on 21 internal Markdown documents, hold multi-turn conversations, securely verify customer identity, and operate via voice using Gnani's speech APIs.

Our Tech Stack:

- **LLM:** Groq (openai/gpt-oss-120b) for high-speed, grounded inference.
- **Embeddings & Vector Store:** Local sentence-transformers/all-MiniLM-L6-v2 with FAISS to eliminate API costs and latency.
- **Speech:** Gnani STT (REST) and Gnani TTS (REST, model timbre-v2.5, voice Kaveri).
- **Framework:** Plain Python, avoiding heavy orchestration frameworks to ensure precise control over tool calls and context injection.

2. Implementation by Step

Step 1: LLM Prompting (The Baseline)

We created a SupportAgent class that connects to the Groq API. We initialized a system prompt instructing the LLM to act as a ZentroMart assistant. At this stage, it had no access to the company documents and relied entirely on its pre-trained knowledge, which led to frequent hallucinations regarding specific return policies and dates.

Step 2: RAG (Grounding in Documents)
We built an ingestion pipeline that chunked the kb/*.md files by their [SECTION-ID] headers. We preserved crucial metadata: doc_type (policy, agent guide, help, faq, marketing) and status (current vs. superseded). The FAISS index retrieved the top-6 chunks, and we enforced a precedence rule to prioritize current policies over marketing pages.

Step 3: Conversational Agent & Tools
We added state management to track the order_id across multiple turns. We wrapped the provided tools.py mock tools. To enforce security, the agent checks for an 8-digit order ID and a 4-digit or 6-digit verification value (phone suffix or pincode). We implemented a hard Python override: if the tool returns verified: True, the Python code bypasses the LLM entirely to guarantee the response, *"Thank you, your identity is verified..."* (preventing the LLM from re-asking for details). If the user requests something out of policy (e.g., compensation), the agent creates a support ticket.

Step 4: Voice Integration (Gnani)
We created a VoiceAgent wrapper. Gnani STT (en-IN) transcribed the customer audio. Because STT models transcribe numbers as words (e.g., "six nine five"), we wrote a custom Python normalizer to convert spoken digit strings into numeric strings (e.g., "695") before passing them to the agent. Finally, Gnani TTS synthesized the agent's reply into a .wav file.

3. Evaluation: Step 1 vs Step 2 (RAG)

We tested the agent on the 47 "answerable" practice questions from the dataset, comparing the baseline Prompting (Step 1) against our RAG implementation (Step 2).

Results:

- **Step 1 (Prompting Only):** 14.9% (7/47 correct)
- **Step 2 (RAG):** 23.4% (11/47 correct)

What went wrong and what we learned:

1. **The Strict Keyword Matcher:** Our automated script required exact factual matches (e.g., 2026-10-27). The LLM frequently answered in natural language (e.g., "27th October"), causing false negatives in our score.
2. **Language Barrier:** 10 of the 47 questions were in Hindi or Hinglish. Our local embedding model (MiniLM) is English-only. It failed to retrieve the correct English documents when queried in Hindi.
3. **Multi-Step Logic:** Questions asking *"If my refund started on 12 October, when will it arrive?"* require date math. The RAG retrieved the policy, but the LLM failed to calculate the business days correctly.

What we would fix:

- **Multilingual Embeddings:** Swap to paraphrase-multilingual-MiniLM-L12-v2 to handle Hindi/Hinglish queries.
- **Function Calling for Dates:** Give the LLM a calculate_business_days Python tool rather than asking it to do math internally.
- **Voice Optimization:** Our voice pipeline successfully bypassed the RAG/LLM for verification steps by extracting numbers directly from the STT transcript.

4. Demo Walkthrough & Handover

Our 2-minute demo video demonstrates a user calling in via voice.

1. The user speaks: *"Hi, my order is six nine five two zero eight six seven and my pin code is one six zero zero one seven."*
2. Gnani STT transcribes the audio.
3. Python normalizes the text to numbers.

4. The agent securely verifies the order (which is shipped) and informs the user it cannot be cancelled, but offers the return policy.
5. **Handover:** When the user asks for "£500 compensation," the agent refuses and creates a support ticket (create_ticket), handing over to a human agent.